

Production Deployment Guide

How to take the Agentic Research Agent from a local CLI package to a reliable, observable, and secure production service — covering the application changes required, the deployment strategies available, and the industry practices used to run LLM agents in production.

Document type: Deployment & Operations Strategy

Generated: 2026-06-05

Project: agentic-ai-learning

Audience: Platform / MLOps / Backend

Contents

- | | |
|---|---|
| 1. Purpose & Scope | 7. Industry Best Practices for Agents |
| 2. Production Readiness Assessment | 8. CI/CD with Evaluation Gates |
| 3. Required Application Changes | 9. Scaling & Capacity Planning |
| 4. Target Production Architecture | 10. Monitoring, SLOs & Alerting |
| 5. Deployment Strategies | 11. Security & Compliance |
| 6. Release & Rollout Strategies | 12. Phased Production Roadmap |

1. Purpose & Scope

The agent today is a well-structured Python package exposing a `research-agent` CLI and a `ResearchAgent` facade. That is production-grade for *batch, worker, and internal-tool* use. To serve real users concurrently over a network — the most common production target — the application needs a thin set of additions around its already-clean core. This guide describes those additions, then the strategies and practices to operate them.

Already production-shaped

Layered package, provider-agnostic LLM factory, typed/validated config, structured + JSON logging, run IDs & durations, tool-call resilience, tests/lint/types green.

Needs adding for a service

HTTP API, durable conversation state, health/readiness probes, authn/z, rate limiting, distributed tracing, and a shared (not file-local) vector store.

Guiding principle

Treat the LLM agent like any other stateful network service: make it stateless, externalize its state, bound every external call, and measure everything.

2. Production Readiness Assessment

A gap analysis of the current codebase against the requirements of a multi-user production service. Severity reflects impact on a networked deployment; for batch/CLI use most items are not applicable.

Area	Current state	Gap	Severity
Interface	Typer CLI + library facade	No HTTP/RPC surface for concurrent remote callers	High
Conversation state	In-process MemorySaver	Lost on restart; not shared across replicas — breaks horizontal scaling	High
Vector store	Local Chroma file under data/vector_store	Not safely shared by multiple replicas; tied to a node's disk	High
AuthN / AuthZ	None	No identity, quotas, or per-tenant isolation	High
Health checks	None	Orchestrators cannot determine liveness/readiness	Medium
Rate limiting / quotas	None	One caller can exhaust provider tokens/budget	Medium
Tracing / metrics	Structured logs + run metadata only	No span-level tracing of LLM/tool calls; no metrics endpoint	Medium
Resilience	Tool-call recovery + LLM timeout	No retry/backoff on 429s, no circuit breaker, no fallback model	Medium
Cost controls	max_tokens, recursion_limit	No per-request budget, caching, or model routing	Medium
Guardrails	Calculator hardening; safe tools	No input/output moderation or prompt-injection defense	Medium

Area	Current state	Gap	Severity
		for untrusted users	
Packaging	uv + hatchling; py.typed	No Dockerfile/manifests in repo (runbook shows a pattern)	Low
CI/CD	Local test/lint/type gates	No automated pipeline or eval gates	Low

Bottom line: the core is sound. The work is additive — a service shell, externalized state, and operational controls — not a rewrite. None of the changes below touch the agent's reasoning logic.

3. Required Application Changes

The most impactful, concrete changes, in priority order. All are additive and preserve the existing `ResearchAgent` contract.

3.1 Add an HTTP API layer (FastAPI + ASGI server)

Wrap the facade in a thin, async-capable API. Build the agent **once** at startup (it loads an embedding model and compiles the graph — expensive), then share it across requests. Never construct `ResearchAgent()` per request.

```
# src/agentic_research_agent/api/app.py
from contextlib import asynccontextmanager
from fastapi import FastAPI, Depends, HTTPException
from agentic_research_agent import ResearchAgent
from agentic_research_agent.schemas.models import AgentRequest, AgentResponse

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Build once: loads embeddings, compiles graph, opens checkpointer.
    app.state.agent = ResearchAgent()
    yield
    # (close pools / flush traces here)

app = FastAPI(title="Research Agent API", version="0.1.0", lifespan=lifespan)

@app.post("/v1/ask", response_model=AgentResponse)
async def ask(req: AgentRequest, _=Depends(verify_api_key)):
    try:
        # ask() is blocking; FastAPI runs sync deps in a threadpool.
        return app.state.agent.ask(req.question, thread_id=req.thread_id)
    except Exception as exc:
        raise HTTPException(status_code=502, detail="agent_run_failed") from exc
```

Serve with `uvicorn` behind `gunicorn` for multiple workers:

```
gunicorn agentic_research_agent.api.app:app \
    -k uvicorn.workers.UvicornWorker -w 4 -b 0.0.0.0:8000 --timeout 120
```

Concurrency caveat: the local sentence-transformers model and a file-based Chroma store are not guaranteed thread/process-safe under heavy parallelism. Pair the API with §3.3 (shared vector DB) and bound worker count, or move embeddings to a dedicated service.

3.2 Durable, shared conversation state (replace `MemorySaver`)

`MemorySaver` keeps thread state in one process's RAM — it is lost on restart and invisible to other replicas. For any multi-instance or restart-surviving deployment, use a durable checkpointer. The `service.py` already isolates this in one line.

```
# add dependency: langgraph-checkpoint-postgres
from langgraph.checkpoint.postgres import PostgresSaver

# in ResearchAgent.__init__, choose by config:
if settings.checkpointer == "postgres":
    cp = PostgresSaver.from_conn_string(settings.postgres_dsn)
    cp.setup()                    # idempotent: creates tables
    self._checkpointer = cp
else:
    self._checkpointer = MemorySaver()  # dev/test default
```

Add `checkpointer` and `postgres_dsn` fields to `Settings`. Use `SqliteSaver` for single-node persistence, `PostgresSaver` for multi-replica.

3.3 Externalize the vector store

A local Chroma file cannot be shared cleanly by multiple replicas and ties the service to one node's disk. Options, in increasing operational maturity:

Option	Fit
Chroma in client/server mode	Minimal change; one shared Chroma server.
<code>pgvector</code> (Postgres extension)	Reuse the same Postgres as the checkpointer — one datastore to operate.
Qdrant / Weaviate / Milvus	Purpose-built, scalable self-hosted vector DBs.

Option	Fit
Pinecone / managed	Fully managed; least ops, usage-based cost.

Keep ingestion (§ runbook) as a **separate pipeline/job** that writes to the shared store; the serving API should only read.

3.4 Health & readiness probes

```
@app.get("/health/live")          # process is up
def live(): return {"status": "ok"}

@app.get("/health/ready")         # dependencies are reachable
def ready():
    checks = {
        "vector_store": app.state.agent.knowledge_base_ready(),
        "llm": app.state.agent.llm_reachable(), # cheap ping / cached
    }
    ok = all(checks.values())
    return JSONResponse({"ready": ok, "checks": checks}, status_code=200 if ok else 500)
```

Liveness restarts a wedged process; readiness gates traffic until the embedding model is loaded and the store/provider are reachable.

3.5 Configuration, secrets & 12-factor

- Keep all config in environment variables (already the case via `pydantic-settings`). Add `API_KEYS` / `JWT` settings, `POSTGRES_DSN`, `VECTOR_DB_URL`, `checkpointer`, `vector_backend`.
- Inject secrets at runtime from a manager (AWS Secrets Manager, GCP Secret Manager, Vault, K8s Secrets) — never bake into images or commit `.env`.
- Maintain per-environment profiles (dev / staging / prod) that differ only by env values.

3.6 Observability: distributed tracing & metrics

Structured logs and run metadata exist; add span-level tracing and metrics so you can see *inside* a run (which tool was slow, how many tokens, retries).

- **LangSmith** — set `LANGSMITH_TRACING=true` and `LANGSMITH_API_KEY`; LangChain/LangGraph auto-trace every LLM and tool call. (LangSmith is already a

dependency.)

- **OpenTelemetry** — instrument the API and export traces/metrics to your collector (Jaeger, Tempo, Datadog).
- **Prometheus metrics** — expose `/metrics`: request rate, p50/p95/p99 latency, error rate, tokens/run, tool-call counts, provider 429s.

3.7 Resilience: timeouts, retries, circuit breaking, fallback

The agent has an LLM timeout and tool-call recovery. Add the rest of the resilience stack:

- **Retry with exponential backoff + jitter** on provider 429/5xx (e.g. `tenacity`, already available).
- **Circuit breaker** per external dependency (LLM, search) to fail fast during an outage.
- **Fallback model** — route to a secondary provider/model when the primary is degraded.
- **Hard request timeout** at the API edge so a stuck run cannot hold a worker forever.

3.8 Guardrails & safety (untrusted input)

- Validate/limit input length and reject obviously malicious payloads at the edge.
- Add prompt-injection mitigation for RAG/web content (treat retrieved text as data, not instructions; spotlighting/delimiting).
- Optional input/output moderation (provider moderation API, Llama Guard, or a guardrails library).
- Principle of least privilege for tools — the current tools are read-only/compute-only; keep any future write/network tools behind explicit policy checks.

3.9 Cost controls

- Per-request and per-tenant **token budgets**; reject or truncate beyond budget.
- **Caching** — semantic cache for repeated questions; provider prompt caching for stable system prompts.
- **Model routing/cascade** — cheap model first, escalate only when needed.

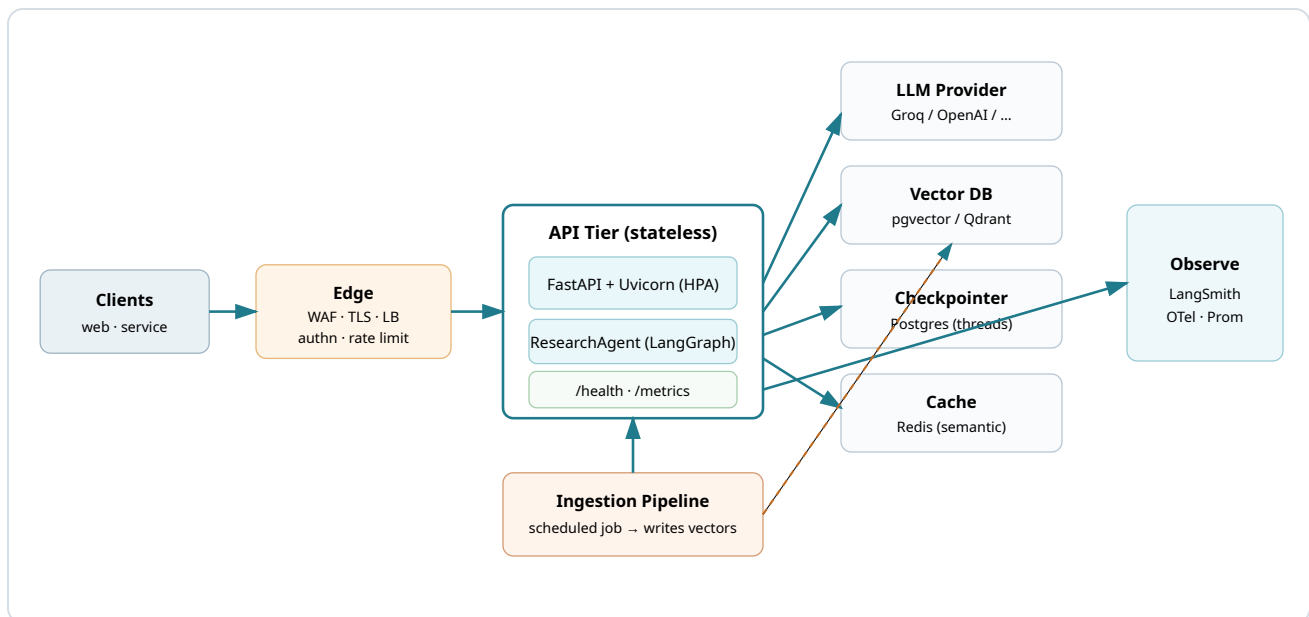
3.10 Container & release artifacts

Add a multi-stage `Dockerfile` (the runbook has a baseline), bake or pre-warm the embedding model, and ship the vector index as a versioned artifact or build it via the

ingestion job — not at container start.

4. Target Production Architecture

A horizontally-scalable, stateless API tier with all state pushed to managed/shared backends and every external call observed.



Stateless API replicas autoscale horizontally; conversation memory, vectors, and cache live in shared backends. Ingestion runs out-of-band and only writes to the vector DB (dashed). Every external call is traced.

5. Deployment Strategies

From lowest to highest operational maturity. Choose based on traffic shape, team capability, and latency/availability requirements.

5.1 Batch / Worker / Scheduled job (supported today)

Run the CLI or import the facade inside a cron job, Airflow/Dagster task, or queue consumer. No API needed. Ideal for periodic research digests, offline enrichment, or internal automation.

5.2 Single container on a VM (Docker Compose)

API container + Postgres + a vector DB via Compose. Simplest always-on service; fits low traffic and demos. Single node = limited availability.

```
services:
  api:
    image: agentic-research-agent:0.1.0
    command: gunicorn agentic_research_agent.api.app:app -k uvicorn.workers.UvicornWorker
    env_file: .env.production
    depends_on: [postgres]
  postgres:
    image: pgvector/pgvector:pg16
    environment: { POSTGRES_PASSWORD: ... }
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes: { pgdata: {} }
```

5.3 Kubernetes (self-managed orchestration)

Deployment + HPA + Service + Ingress, with secrets and probes. The standard for scalable, self-operated services.

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: research-agent }
spec:
  replicas: 3
  template:
```

```

spec:
  containers:
    - name: api
      image: registry/agentic-research-agent:0.1.0
      ports: [{ containerPort: 8000 }]
      envFrom: [{ secretRef: { name: research-agent-secrets } }]
      readinessProbe: { httpGet: { path: /health/ready, port: 8000 }, initialDelaySeconds: 10 }
      livenessProbe: { httpGet: { path: /health/live, port: 8000 } }
      resources: { requests: { cpu: "500m", memory: "1Gi" }, limits: { cpu: "2", memory: "4Gi" } }
  ---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: research-agent }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: research-agent }
  minReplicas: 3
  maxReplicas: 20
  metrics: [{ type: Resource, resource: { name: cpu, target: { type: Utilization, averageValue: 70 } } }]

```

5.4 Serverless containers (Cloud Run / ECS Fargate / Azure Container Apps)

Push the image; the platform handles autoscaling (incl. scale-to-zero), TLS, and rollout. Lowest ops for a real API. Watch cold starts — the embedding-model load makes startup heavy, so set a warm `minInstances ≥ 1` and a generous startup probe.

5.5 Function-as-a-Service (Lambda / Cloud Functions)

Possible but awkward for this workload: large dependency bundle (torch/sentence-transformers), cold-start cost, and per-invocation time limits. If used, externalize embeddings entirely and keep functions thin. Generally **not recommended** as the primary tier here.

5.6 Managed agent platform (LangGraph Platform)

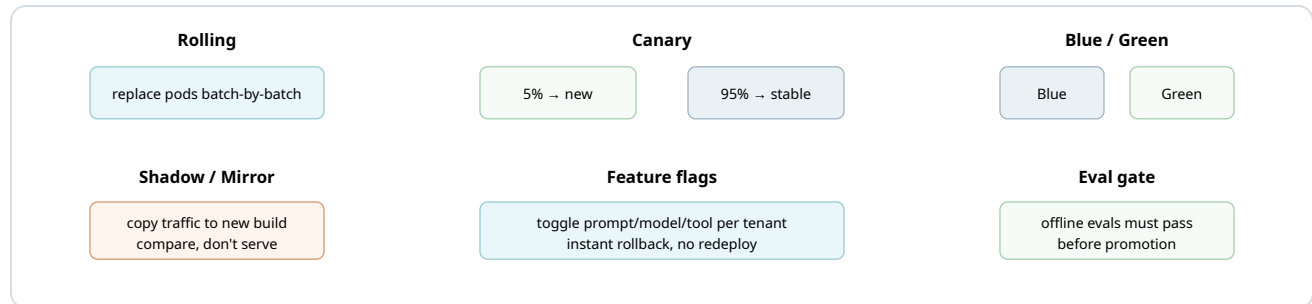
Because the agent is already a compiled LangGraph, it can be deployed to a managed LangGraph runtime that provides persistence, streaming, scaling, and a task queue out of the box — trading control for speed-to-production.

Decision matrix

Strategy	Ops effort	Scale	Best for
Batch / worker	Very low	N/A	Offline jobs, internal tools (today)
Compose on VM	Low	Single node	Demos, low traffic, pilots
Serverless container	Low-med	Auto ($\rightarrow 0$)	Bursty/variable traffic, small teams
Kubernetes	High	High	Steady high traffic, platform teams
Managed LangGraph	Low	Managed	Fastest path; accept platform lock-in

6. Release & Rollout Strategies

Agent quality is probabilistic, so prefer rollout patterns that expose changes to a fraction of traffic and compare behavior before full cutover.



Pattern	How it helps an agent
Rolling	Zero-downtime default; gradual replacement with health gating.
Canary	Route a small % to the new prompt/model/build and watch quality + latency + cost before ramping.
Blue/Green	Instant, atomic cutover and instant rollback — keep the old version warm.
Shadow / mirror	Send real traffic to the candidate <i>without serving its output</i> ; compare answers offline — ideal for risky prompt/model changes.
Feature flags	Decouple deploy from release; flip prompts/models/tools per tenant and roll back instantly.

7. Industry Best Practices for Agents in Production

What teams running LLM agents at scale converge on, grouped by concern.

Architecture

Stateless services, external state

Keep replicas stateless; push conversation memory to a durable checkpointer so any replica can serve any thread.

Separate ingestion from serving

RAG indexing is a pipeline/job; the serving path only reads. Re-index on a schedule or on document change.

One provider seam

Abstract the model behind a factory (already done) so model/provider swaps are config, enabling routing and fallback.

Async + streaming

Stream tokens to users for perceived latency; use async I/O so a worker isn't blocked during long model calls.

Reliability

Bound every external call

Timeouts on LLM, search, and DB. No unbounded waits.

Retry + backoff + jitter

For 429/5xx; cap attempts; never retry non-idempotent side effects blindly.

Circuit breakers & fallback

Fail fast on a downed dependency; degrade to a fallback model or cached answer.

Cap the loop

A recursion/step limit (already present) plus per-request wall-clock and token budgets stop runaway agents.

Observability & Evaluation

Trace every run

Span-level tracing (LangSmith/OTel) of each LLM and tool call, with token and latency attribution.

Offline evals as a gate

A versioned test set scoring answer quality, retrieval relevance, and citation faithfulness — must pass before promotion.

Online evals & feedback

Sample production traffic for LLM-as-judge scoring; capture thumbs-up/down to build regression sets.

Drift & quality monitoring

Watch for rising refusals, empty tool results, or quality regressions after provider-side model updates.

Safety & Governance

Treat retrieved content as data

Defend against prompt injection from RAG/web text via delimiting/spotlighting and instruction isolation.

Input/output moderation

Screen untrusted input and generated output; redact PII before logging or indexing.

Least-privilege tools

Scope tool permissions tightly; gate any write/network/exec tool behind explicit policy and approval.

Auditability

Persist run_id, inputs, tool calls, and outputs for audit and incident review (with retention rules).

Cost & Performance

Caching

Semantic cache for repeated queries; provider prompt caching for the stable system prompt.

Model routing / cascade

Use a small/cheap model by default; escalate to a larger model only when needed.

Token budgets & quotas

Right-size retrieval

Per-request and per-tenant limits to keep spend predictable.

Tune `top_k` and chunk size; fewer, better chunks cut tokens and improve answers.

Security & LLMOps

Secrets & isolation

Runtime secret injection, network policies, and isolated egress for tool calls.

Version prompts & models

Treat the system prompt and model id as versioned artifacts; tie them to release and evals.

CI/CD with eval gates

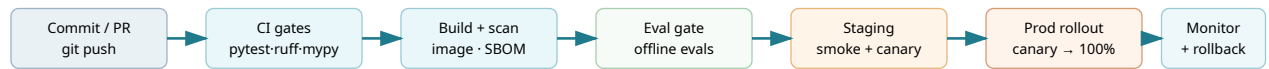
Automate test/lint/type + offline evals + image scan before any promotion.

Supply-chain hygiene

Pin and scan dependencies (`uv.lock`), scan images, and rebuild on CVEs.

8. CI/CD with Evaluation Gates

Extend the existing quality gates with an image build, security scan, an *evaluation* stage unique to LLM apps, and a progressive rollout.



```
# pipeline stages (any CI)
uv sync --frozen
uv run ruff check . && uv run mypy && uv run pytest      # quality gates
docker build -t $REG/agent-research-agent:$SHA .         # build
trivy image $REG/agent-research-agent:$SHA               # vuln + secret scan
uv run python -m evals.run --suite regression --min-score 0.8 # LLM eval gate
# deploy to staging → smoke → canary 5% → ramp → 100%, with auto-rollback on SLO break
```

What's new vs. a normal service: the *eval gate*. Because model/prompt changes can silently degrade quality, a versioned offline eval suite (answer quality, retrieval relevance, citation faithfulness) should block promotion just like a failing unit test.

9. Scaling & Capacity Planning

Dimension	Bottleneck	Strategy
Request throughput	Worker count; LLM latency dominates each request	Horizontal autoscale (HPA/serverless) on CPU + concurrency; keep workers I/O-bound and async.
LLM provider	Provider rate limits / quotas	Request quota increases, queue + backpressure, multi-provider fallback, caching.
Embeddings	CPU-bound local model; slow under load	Move to a dedicated embedding service or hosted embeddings; batch; or GPU nodes for ingestion.
Vector search	Local file store doesn't scale across replicas	Managed/served vector DB with replicas and proper indexing (HNSW params).
State (checkpointer)	Postgres connections per replica	Connection pooling (PgBouncer); size pool to replica count.
Cold starts	Embedding-model load at startup	Warm pool / min instances; readiness probe; bake model into image.

10. Monitoring, SLOs & Alerting

Suggested SLOs

SLI	Example SLO	Source
Availability	99.5% of <code>/v1/ask</code> non-5xx	LB / API metrics
Latency (p95)	< 8 s end-to-end	<code>duration_ms</code> + traces
Run success rate	> 98% non-error runs	<code>agent_run_finished</code> vs <code>agent_run_failed</code>
Cost per request	Within budget; alert on spikes	Token metrics from traces
Answer quality	Online eval score above threshold	LLM-as-judge sampling

Alert on

- Error/failure rate above baseline; provider 401/429 spikes.
- p95 latency breaching SLA; recursion-limit hits rising.
- Empty/failed web-search or retrieval rates climbing.
- Cost-per-request or token-per-run anomalies.
- Readiness failing (vector store / provider unreachable).

The app already emits `agent_run_started/finished/failed` with `run_id`, `duration_ms`, and tool-call counts — a strong foundation. Add tracing and a `/metrics` endpoint to complete the picture.

11. Security & Compliance Checklist

- Secrets injected at runtime from a managed store; `.env` never committed; keys held as `SecretStr` (already done).
- TLS everywhere; WAF and authn/z at the edge; per-tenant authorization.
- Rate limits and quotas per identity to prevent abuse and budget exhaustion.
- Prompt-injection defenses for RAG/web content; treat tool/retrieval output as untrusted data.
- PII handling: redact before logging/indexing; define retention & deletion policy for conversation state.
- Least-privilege, sandboxed tool execution; no arbitrary code/eval (calculator is AST-allow-listed — already done).
- Network egress controls for outbound LLM/search calls; audit logs for every run.
- Dependency & image scanning in CI; pinned `uv.lock`; rebuild on CVEs.

12. Phased Production Roadmap

Crawl — internal / pilot

- Containerize; deploy to a serverless container or single VM.
- Add FastAPI + API-key auth + health probes.
- SQLite/Postgres checkpointer; ship a prebuilt vector index.
- Enable LangSmith tracing; ship JSON logs.

Walk — production v1

- Postgres checkpointer + managed/served vector DB (pgvector/Qdrant).
- Rate limits, retries/backoff, request timeouts, circuit breaker.
- CI/CD with offline eval gate; canary rollout; `/metrics` + dashboards + alerts.
- Semantic + prompt caching; per-tenant token budgets.

Run — scale & harden

- Kubernetes + HPA (or managed LangGraph) with multi-provider fallback.
- Online evals + feedback loop + drift monitoring.
- Guardrails/moderation, full audit, PII redaction, retention policy.
- Model routing/cascade; cost & SLO governance.

Recommended first step: implement §3.1 (FastAPI), §3.2 (durable checkpointer), and §3.4 (health probes), then deploy to a serverless container. That yields a real,

scalable service with minimal code and the lowest operational burden — and nothing in the agent's reasoning core has to change.